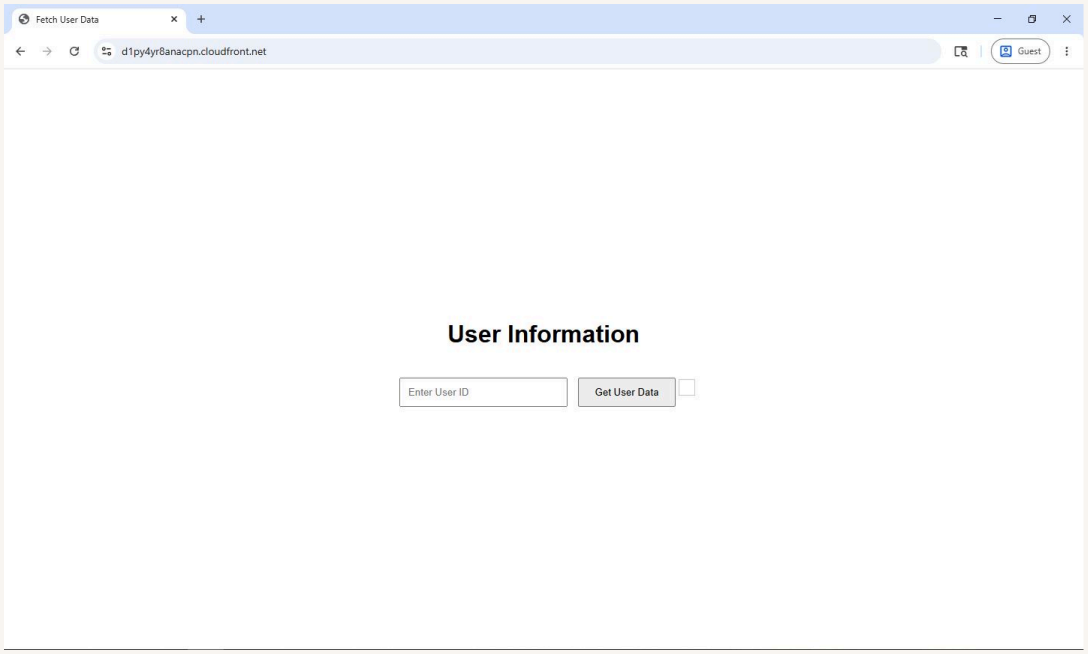
nextwork.org

Website Delivery with CloudFront

Mohammed Dawoud Mota



Introducing Today's Project!

For this project, I will demonstrate how to deploy a website using Amazon S3 and Amazon CloudFront. My goal is to understand how content delivery networks improve website performance and to learn the full process of storing static website files in S3, distributing them globally with CloudFront, and configuring the correct permissions so the site can be delivered securely and efficiently. This project helps me build practical cloud skills while also forming the presentation layer of a three-tier architecture.

Tools and concepts

The key services and concepts I learned in this project were how CloudFront, S3, and an Origin Access Control work together to keep my S3 bucket private while still allowing CloudFront to access my files. I learned how to update and apply the correct S3 bucket policy so only CloudFront can read my objects, how to match the exact bucket name with my CloudFront origin, and how CloudFront needs to be fully deployed before the changes take effect. Overall, the project taught me how to securely deliver content by locking down my bucket and letting CloudFront handle all access.

Project reflection

This project took me a few hours to complete. The most challenging part was creating the OAC and updating the S3 policy to give access to CloudFront. It was most rewarding to see that the web page was accessible.

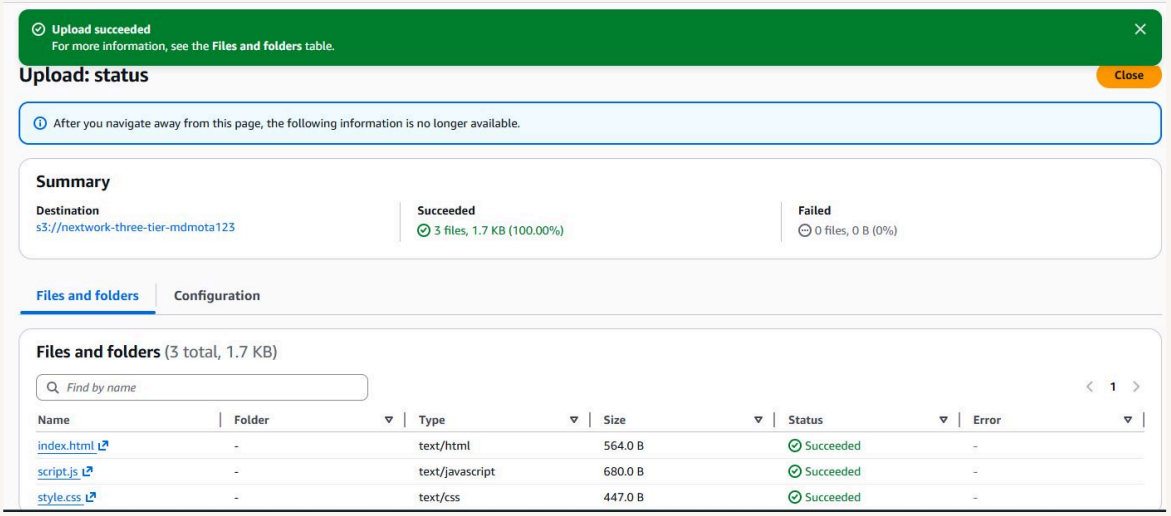
I chose to do this project today because I wanted to build something more complex in the cloud. Something that would make learning with NextWork even better is updating the projects instructions some of them are not updated to the current UI of AWS.

Set Up S3 and Website Files

I used an S3 bucket in this project because it serves as the storage layer for all of my website’s files. CloudFront is designed to distribute content globally, but it cannot store any files on its own. By placing my HTML, CSS, and JavaScript files in S3, I created a reliable and secure origin that CloudFront can pull from when delivering the site to users around the world.

The three files I downloaded were index.html, which contains the main structure and content of the webpage, style.css, which controls the visual design and layout of the site, and script.js, which adds interactivity and handles any client side behavior for the website.

I validated my website files by checking my Downloads folder to make sure all three files were saved correctly, then opening index.html in my browser to confirm that the webpage loaded as expected. Seeing the simple webpage appear told me that the files were intact and ready to be uploaded to S3.

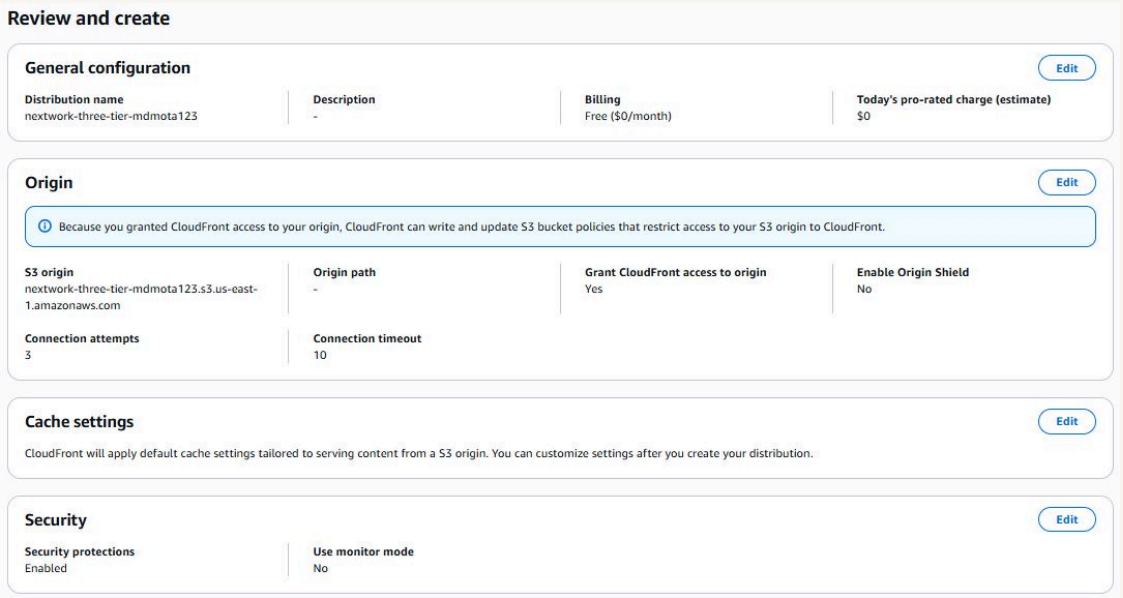

Upload succeeded
For more information, see the Files and folders table.
Upload: status
Close
After you navigate away from this page, the following information is no longer available.
Summary
Destination
s3://nextwork-three-tier-mdmota123
Succeeded
3 files, 1.7 KB (100.00%)
Failed
0 files, 0 B (0%)
Files and folders
Configuration
Files and folders (3 total, 1.7 KB)
Find by name
1
Name
Folder
Type
Size
Status
Error
index.html
-
text/html
564.0 B
Succeeded
-
script.js
-
text/javascript
680.0 B
Succeeded
-
style.css
-
text/css
447.0 B
Succeeded
-

Exploring Amazon CloudFront

Amazon CloudFront is a content delivery network that speeds up how websites deliver files like HTML, CSS, JavaScript and images to users around the world. Businesses and developers use CloudFront because it caches content at edge locations, which reduces latency and improves load times. It also adds security and reliability by serving content through a global network rather than directly from a single origin like an S3 bucket.

A CloudFront distribution is the configuration that tells CloudFront how to deliver my website's content. It defines the origin where my files are stored, how caching should work, and the rules CloudFront follows when serving the site to users. In other words, the distribution acts as the blueprint that controls how CloudFront retrieves, protects and delivers my S3 content across its global network.

I set up my CloudFront distribution by selecting my S3 bucket as the origin and using the recommended settings to keep the configuration simple and secure. I chose the single website or app option, kept the default caching and security settings, and made sure the distribution name matched my S3 bucket for clarity. Once everything looked correct, I created the distribution so CloudFront could begin preparing to deliver my website globally.

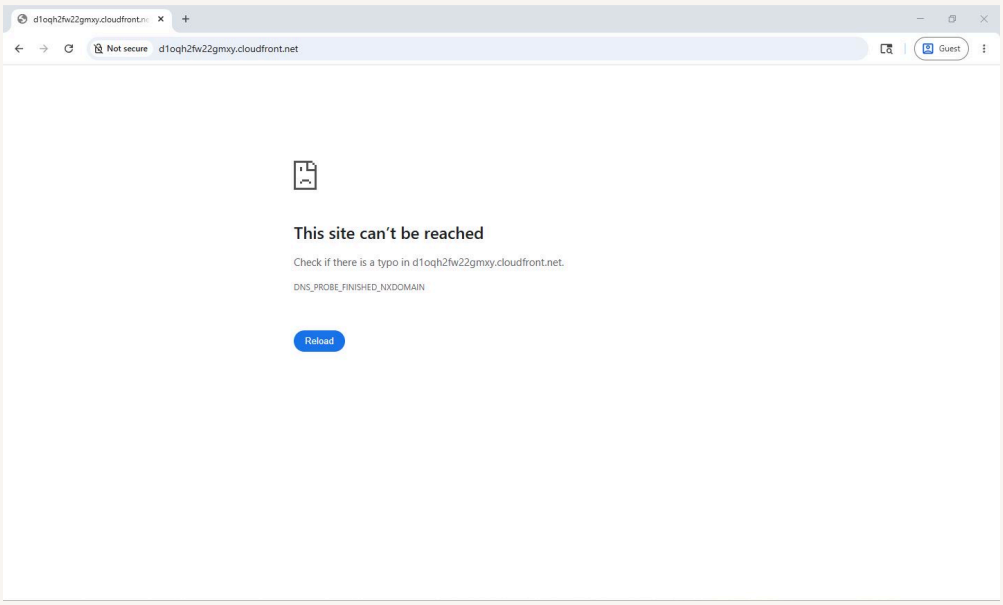

Review and create
General configuration
Edit
Distribution name
nextwork-three-tier-mdmota123
Description
-
Billing
Free ($0/month)
Today's pro-rated charge (estimate)
$0
Origin
Edit
Because you granted CloudFront access to your origin, CloudFront can write and update S3 bucket policies that restrict access to your S3 origin to CloudFront.
S3 origin
nextwork-three-tier-mdmota123.s3.us-east-1.amazonaws.com
Origin path
-
Grant CloudFront access to origin
Yes
Enable Origin Shield
No
Connection attempts
3
Connection timeout
10
Cache settings
Edit
CloudFront will apply default cache settings tailored to serving content from a S3 origin. You can customize settings after you create your distribution.
Security
Edit
Security protections
Enabled
Use monitor mode
No

Handling Access Issues

I ran into the access denied error because even though the origin access setting was initially public, the objects inside my S3 bucket were still private. CloudFront was trying to fetch those files, but the bucket and its contents had not been granted permission for CloudFront to access them. Since S3 keeps all objects private by default, CloudFront could not retrieve the website files, which resulted in the access denied message.

In my distribution, the origin access setting was configured to use the public origin access option, which meant CloudFront expected my S3 bucket to allow public access. But my bucket and all the objects inside it were still private, so CloudFront didn’t actually have permission to read anything. That mismatch between public origin access and private S3 objects is what caused the access denied issue.

Origin Access Control is the feature that lets CloudFront securely access the objects in my S3 bucket without making anything public; it creates a trusted, signed relationship so that only CloudFront can retrieve my files, keeping my bucket private while still allowing my website to load through the distribution.


d1oqh2fw22gmxy.cloudfront.net
Not secure
d1oqh2fw22gmxy.cloudfront.net
Guest
This site can't be reached
Check if there is a typo in d1oqh2fw22gmxy.cloudfront.net.
DNS_PROBE_FINISHED_NXDOMAIN
Reload

Updating S3 Permissions

A bucket policy update is still needed because even though I created the Origin Access Control, S3 doesn't automatically trust it; I have to explicitly grant CloudFront permission to read the objects in my bucket. Until I add that policy, S3 will continue blocking CloudFront's signed requests, which is why the distribution can't serve my website yet.

This policy allows CloudFront to read the objects in my S3 bucket by trusting the signed requests that come from my Origin Access Control, and it keeps everything private by making sure only CloudFront is allowed to access my files while everyone else is blocked.

```
{
        "Version": "2008-10-17",
        "Id": "PolicyForCloudFrontPrivateContent",
        "Statement": [
            {
                "Sid": "AllowCloudFrontServicePrincipal",
                "Effect": "Allow",
                "Principal": {
                    "Service": "cloudfront.amazonaws.com"
                },
                "Action": "s3:GetObject",
                "Resource": "arn:aws:s3:::nextwork-three-tier-mdmota123/*",
                "Condition": {
                    "StringEquals": {
                      "AWS:SourceArn": "arn:aws:cloudfront::289654514139:distribution/E1NXLC2LIY9YCN"
                    }
                }
            }
        ]
    }
```